

U

O

W

CSIT314

Software Development

Methodologies

Kanban and XP



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Agile (recap)

- Project planning that places a premium on flexibility (resource constrain) and evolving customer requirements throughout the development process
- Agile refers to a set of methods and practices
 - the collaborative effort of self-organizing cross-functional teams and their customers and users.
 - adaptive planning, evolutionary development, early delivery, and continuous improvement
 - rapid and flexible response to change



Agile

Agile	Traditional (Waterfall)
Working software	Comprehensive documentation
Customer collaboration	Contract negotiation
Responding to change	Following a plan
Individuals and interactions	Process and tools

- Scrum
- Kanban
- Extreme Programming



Kanban - history

- Kanban (meaning “billboard” or “signboard” in Japanese)
- Kanban was originally developed by Toyota for manufacturing
- Adapted to software development in 2010

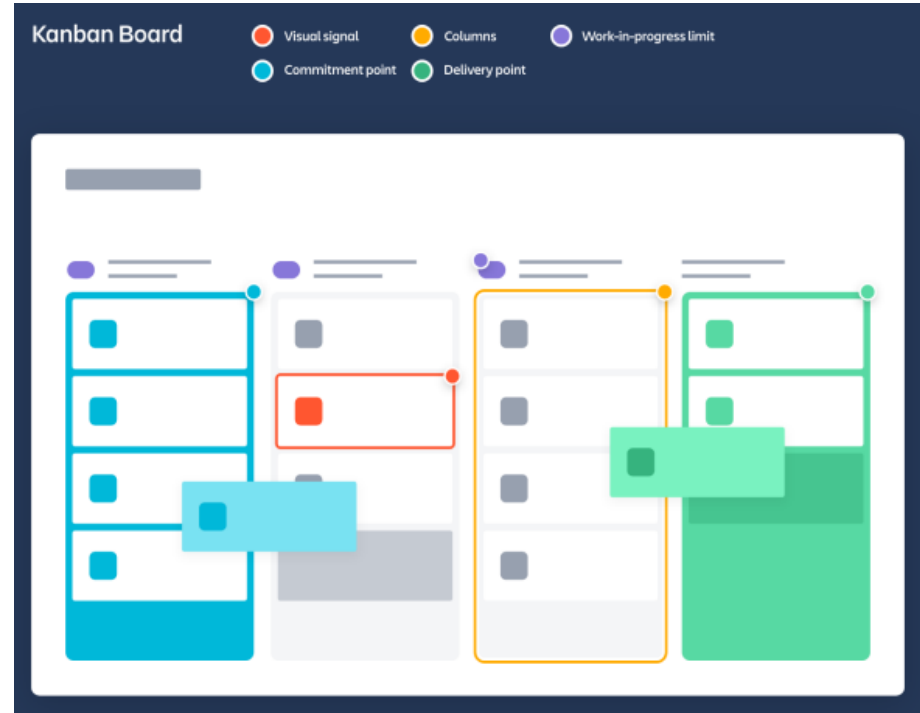
Kanban

- A popular agile software development methodology/framework
- Real-time team communication
- Full transparency of work
- Visually tracking progress



Kanban board

- Represents the overall project
- Has 5 components:
 - Visual signals
 - Columns
 - Work-in-progress limits
 - Commitment point
 - Delivery point



Kanban board (cont.)

- Visual signals:
 - Visual cards representing work items or user stories
 - These cards help all team members quickly see what the team is working on
- Columns:
 - Each column represents different “stage” in the workflow
 - Cards go through each stage in the workflow
 - Workflows can be customized:
 - “To Do” , “In Progress” , “Code Review” , “Done”



Kanban board (cont.)

- Work In Progress (WIP) Limits:
 - Maximum number of cards in one column at any given time
 - When the maximum number of cards is reached, the team needs to focus on the work in those cards to move them to the next stage
 - Once this is resolved, new cards can be moved into that column
 - This is a quick way to identify bottlenecks



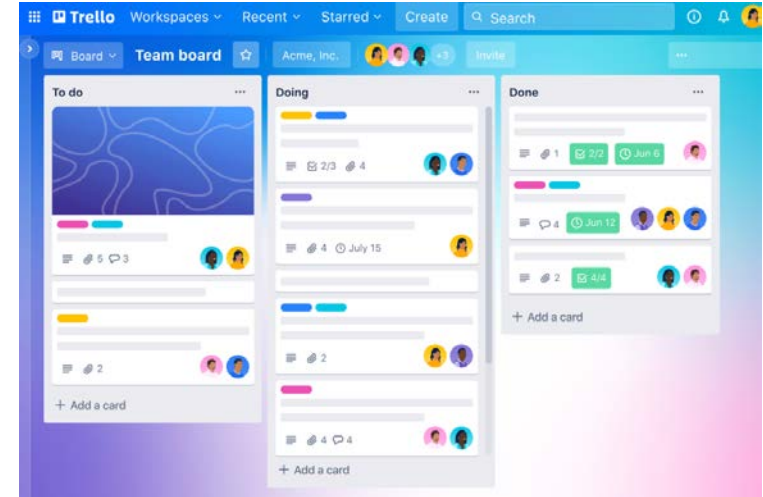
Kanban board (cont.)

- Commitment point:
 - The time point when an item is selected from the backlog and moved into the Kanban board
 - Signifies that work for this item is now started
- Delivery point:
 - The end of a Kanban team' s workflow
 - Product is delivered to the clients
 - Lead time (between a request being made and a task being released) = Delivery point - Commitment point



Example

- Physical and virtual



Example

- Trello
 - <https://youtu.be/13F313psqXY>
- Taiga
 - <https://www.youtube.com/watch?v=6VsSvoJHiAk>



Kanban benefits

- Easy to use, simplicity
- Visual interface
- Instantly see what everyone is working on
- Visibility in terms of task progress and bottlenecks (e.g. a specific task is holding up the project)



Kanban vs. Scrum

	Scrum	Kanban
Cadence	Regular fixed length sprints (ie, 1-4 weeks)	Continuous flow
Release methodology	The end of each sprint	Continuous delivery
Roles	Product owner, scrum master, development team	No required roles
Key metrics	Velocity	Lead time, cycle time, WIP
Change philosophy	Teams should not make changes during the sprint.	Change can happen at any time



Case study

- Background
 - Mobile.de, an online search tool for buying and selling vehicles
 - From 1 star rating (2010) to 4 stars rating (2013) in Apple store
 - iPad, Android apps, and a mobile website portal (generate 35% of the overall traffic for the service)
 - With the help of Kanban method and lean thinking, they put the app at the top of the mobile rankings for Germany



Extreme programming

- One several popular agile methods
- Key characteristics:
 - Stress customer satisfaction
 - Empower developers to confidently respond to changing requirements
 - Strong emphasis on teamwork and peers
 - Deliver as early as possible and obtain early feedback from customers



Extreme programming

- XP incorporates the following 5 values
 - Communication: XP focuses on effective communication between the team and the customer
 - Simplicity: XP looks for the simplest ways to get things done. This means doing what is essential
 - Feedback: XP encourages instantaneous feedback
 - Respect: The team must respect each other
 - Courage: XP endorses courage at all levels



The Values of Extreme Programming

- Communication:
 - Everyone is part of the team and we communicate face to face daily
 - We will work together on everything from requirements to code
 - We will create the best solution to our problem that we can together.



The Values of Extreme Programming

- Simplicity
 - We will do what is needed and asked for, but no more
 - We will take small simple steps to our goal and mitigate failures as they happen
 - We will create something we are proud of and maintain it long term for reasonable costs



The Values of Extreme Programming

- Feedback:
 - We will take every iteration commitment seriously by delivering working software
 - We demonstrate our software early and often then listen carefully and make any changes needed
 - We will talk about the project and adapt our process to it, not the other way around



The Values of Extreme Programming

- Respect:
 - Everyone gives and feels the respect they deserve as a valued team member
 - Everyone contributes value even if it's simply enthusiasm
 - Developers respect the expertise of the customers and vice versa
 - Management respects the right to accept responsibility and receive authority over the work



The Values of Extreme Programming

- Courage:
 - We will tell the truth about progress and estimates
 - We don't document excuses for failure because we plan to succeed
 - We don't fear anything because no one ever works alone
 - We will adapt to changes when ever they happen.



XP Practices

Group	Practices
Feedback	✓ Test-Driven Development ✓ The Planning Game ✓ On-site Customer ✓ Pair Programming
Continual Process	✓ Continuous Integration ✓ Code Refactoring ✓ Small Releases
Code understanding	✓ Simple Design ✓ Collective Code Ownership ✓ System Metaphor ✓ Coding Standards
Work conditions	✓ 40-Hour Week

- XP suggests using 12 practices while developing software which can be clustered into four groups.

XP Practices: Feedback

- **Test-Driven Development:** XP teams practice test-driven development technique (TDD) that entails writing an automated unit test before the code itself. Every piece of code must pass the test to be released. TDD allows programmers to use immediate feedback to produce reliable software
- **The Planning Game:** This is a meeting that occurs at the beginning of an iteration cycle. The development team and the customer get together to discuss and approve a product's features
- **On-site Customer:** The customer should be present all the time to answer team questions, set priorities, and resolve disputes if necessary.
- **Pair Programming:** This practice requires two programmers to work jointly on the same code. While the first developer focuses on writing, the other one reviews code, suggests improvements, and fixes mistakes along the way.



XP Practices: Continue Process

- **Continuous Integration:** Developers always keep the system fully integrated. XP teams take iterative development to another level because they commit code multiple times a day
- **Code Refactoring:** Refactoring is about removing redundancy, eliminating unnecessary functions, increasing code coherency, and at the same time decoupling elements. Keep your code clean and simple, so you can easily understand and modify it when required would be the advice of any XP team member
- **Small Releases:** This practice suggests releasing the MVP quickly and further developing the product by making small and incremental updates. Small releases allow developers to frequently receive feedback, detect bugs early, and monitor how the product works in production.



XP Practices: Code Understanding

- **Simple Design:** The best design for software is the simplest one that works. If any complexity is found, it should be removed. The right design should pass all tests, have no duplicate code, and contain the fewest possible methods and classes.
- **Collective Code Ownership:** This practice declares a whole team's responsibility for the design of a system. Each team member can review and update code.
- **System Metaphor:** System metaphor stands for a simple design. The team should be able to start working on it without spending too much time examining specifications.
- **Coding Standards:** A team must have common sets of coding practices, using the same formats and styles for code writing.



XP Practices: Work Condition

- **40-Hour Week:** XP projects require developers to work fast, be efficient, and sustain the product's quality. To adhere to these requirements, they should feel well and rested. Keeping the work-life balance prevents professionals from burnout. In XP, the optimal number of work hours must not exceed 45 hours a week.

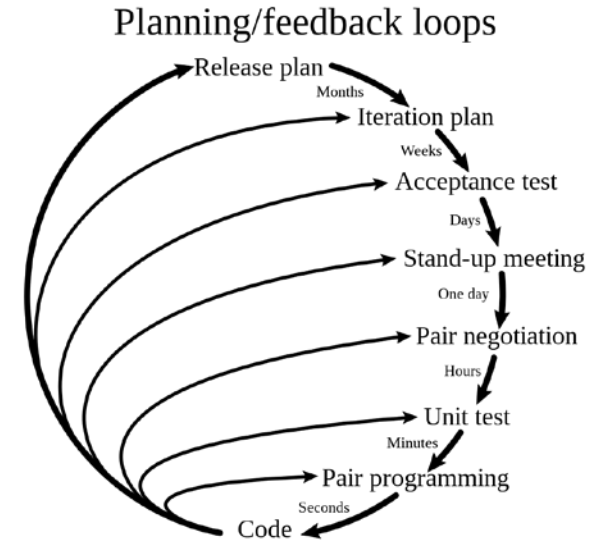
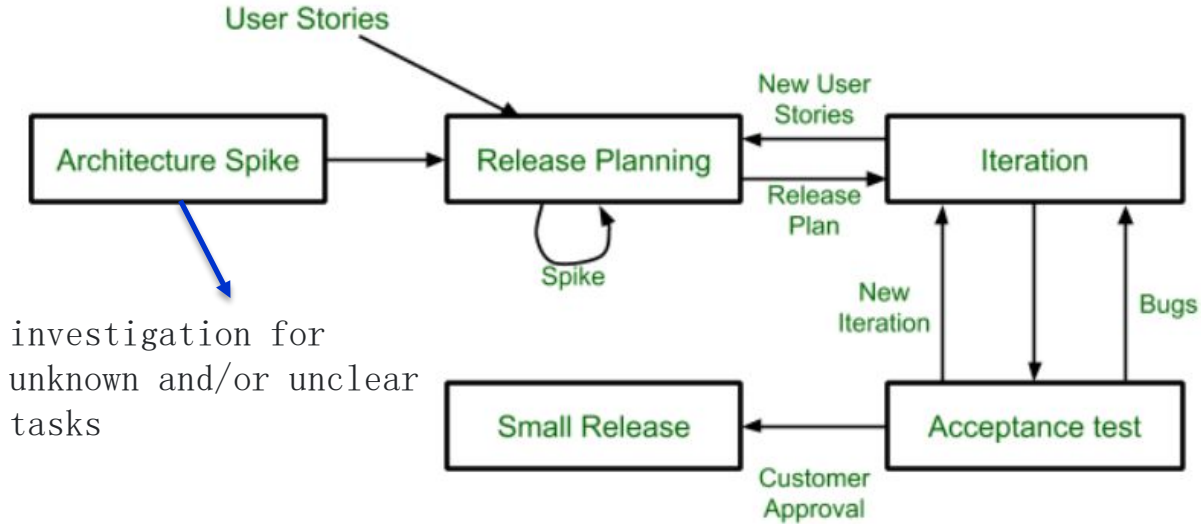


Extreme Programming Participants

- **Customers** are expected to be heavily engaged in the development process by creating user stories, providing continuous feedback, and making all the necessary business decisions related to the project.
- **Developers** are the team members that actually create the product. They are responsible for implementing user stories and conducting user tests.
- **Trackers** link customers and developers. It's not a required role and can be performed by one of the developers. These people organize the meetups, regulate discussions, and keep track of important progress.
- **Coaches** can be included in the teams as mentors to help with understanding the XP practices. It's usually an external consultant who is not involved in the development process, but has used XP before and so can help avoid mistakes.



Extreme Programming Process



Spike

- Create spike solutions to reduce risk
 - spike solution is a very simple program to explore potential solutions
 - build the spike to only addresses the problem under examination and ignore all other concerns
 - Most spikes are not good enough to keep, so expect to throw it away
 - Refactor whenever and wherever possible



Release planning

- User stories are written
- Each user story represents a desired feature and all stories represent the specification of the system
- Customers prioritise user stories based on business value (e.g. essential, highly valuable or good idea)
- Estimate each user story in terms of ideal days

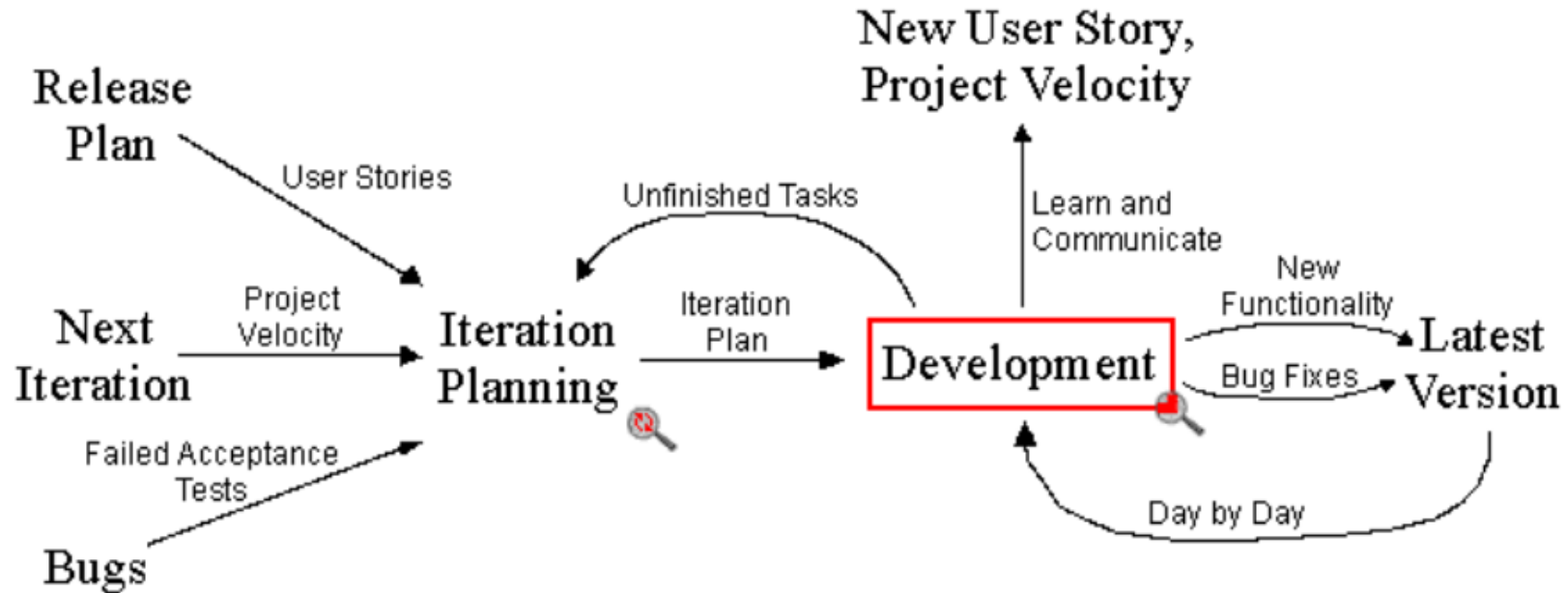


Release planning

- Together developers and customers create a set of stories to be implemented as the first (or next) release
- Make frequent small releases
 - A (re)useable, testable system that makes good business sense delivered early is desired
- You may plan by time or by scope:
 - how many stories can be implemented before a given date (time)
 - how long a set of stories will take to finish (scope)



Iteration



Iteration planning

- Iteration planning takes place before each iteration
- Unimplemented user stories are selected from the iteration plan for the upcoming iteration
- User stories implemented in the previous iterations but did not pass acceptance tests are also selected
- The total user stories selected should have the total estimates up to the project velocity from the last iteration
 - how much work is getting done on your project
 - simply adding up the estimates of the user stories that were finished during the iteration



Designing

- Simplicity in design
 - Testable, Understandable, Browsable, and Explainable
- Use Class Responsibilities and Collaboration (CRC) cards for design sessions.

Class Name	
Responsibilities	Collaborators

Student	
Student number Name Address Phone number Enroll in a seminar Drop a seminar Request transcripts	Seminar

Coding

- Code must be written to agreed coding standards (different languages)
- Code the unit test first (Test Driven Development)
- All production code is pair programmed
- Code review
- Integrate often



Coding (for test)

- All code must have unit tests
- All code must pass all unit tests before it can be released
- When a bug is found, tests are created
- Acceptance tests are run often
 - created from user stories
 - customer specifies scenarios to test when a user story has been correctly implemented
 - one story can have one or many acceptance tests
 - acceptance tests are black box system tests
 - a user story is not considered complete until it has passed its acceptance tests



Managing

- Give the team a dedicated open work space
- Set a sustainable pace
- A stand up meeting starts each day
- Move people around



Advantages and Disadvantages of XP

- Continuous testing and refactoring practices help create stable well performing systems with minimal debugging
- Simplicity value implies creating a clear, concise **code that is easy to read and change** in the future if needed
- The workable results can be **delivered very soon** and only necessary features are built
- **Documentation is reduced** as bulky requirement documents are substituted by user stories
- **Constant communication** provides a high level of visibility and accountability
- **Pair programming** is higher-quality products with fewer bugs
- **Customer engagement** ensures their satisfaction as their participation in the development and testing process



Advantages and Disadvantages of XP

- The customer has **no clear picture** of the end result, which makes it almost unrealistic to accurately estimate scope, cost, and time
- **Regular meetings** with customers often take a great deal of time that could instead be spent on actual code writing
- **Lacking clear requirements** and specifications, leading to project scope creep
- The transition from **traditional methods** to **extreme programming** demands significant cultural and structural changes
- **Pair programming** takes more time and doesn't always work right due to the human factor and character incompatibility
- XP works best with collocated teams and customers present in person to conduct face-to-face meetings, limiting its application with **distributed teams**
- Sometimes customers have **neither the desire, time, nor expertise** to participate in product development
- Over-focusing on **code over design**, lack of quality assurance, code duplication, and poor results with inexperienced developers.



XP vs Scrum

- Scrum is commonly associated with self-organizing teams. It also usually has sprints that are 2 to 4 weeks long, while XP iterations are shorter, taking 1 to 2 weeks
- Besides, XP is much more flexible with possible changes within iterations, while Scrum doesn't allow any modifications after the sprint backlog is set
- in XP the customer prioritizes features and decides on the order of their development, but in Scrum the team itself determines what to work on first
- Scrum's main roles are Product Owner, Scrum Master, and Scrum Team, which are different from those in XP
- Incorporating XP practices and Scrum techniques is considered quite effective with XP focusing on engineering aspects and Scrum organizing the process.



XP vs Kanban

- Kanban puts a lot of focus on visualizing the development process and strictly limits the number of features developed at a time
- It' s also characterised by a continuous workflow while XP has separate iterations, even though both suggest small frequent releases and a high level of flexibility and adaptiveness to the changing requirements
- The roles in Kanban are not strictly defined.



When to use XP

Team's size, structure, and expertise

- **Highly-adaptive development.** development teams adapt to fast-changing requirements
- **Risky projects.** to avoid problems connected with working on a new system, especially when a customer sets strict deadlines for a project
- **Small teams.** XP practices are efficient for teams that don't exceed 12 people
- **Automated testing.** XP requires the developers' ability to create and run unit tests, and availability of the necessary testing tools
- **Readiness to accept new culture and knowledge.** It's important to embrace change, and have an experienced coach
- **Customer participation.** work side-by-side, make sure your client is always available to provide input until a project ends.

